Mini Juego Piedra, Papel o Tijera con Persistencia en Disco

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 7 — Mini juego «Piedra, papel o tijera» con persistencia de datos **Versión del documento:** 1.0.0 **Fecha:** 2026

Introducción

Muchos programadores junior confunden la **Memoria RAM** (volátil: los datos se borran al cerrar el programa) con el **Almacenamiento Secundario o Disco** (no volátil: archivos de texto persistentes). Esa confusión es la causa de fondo de la mayoría de los errores de «mi programa no guardó nada».

En esta actividad se desarrolló un **Mini Juego de Piedra, Papel o Tijera contra la Inteligencia Artificial de la Consola**, donde cada partida jugada se registra automáticamente en una estructura (`struct`) y se guarda físicamente en un archivo de texto (`historial_partidas.txt`) en el disco.

Los objetivos concretos fueron: modelar la partida con `struct`, escribir en disco con `ofstream` en modo `Append` (`ios::app`) para no sobrescribir las partidas previas, y leer el historial completo desde el disco con `ifstream` para desplegarlo en pantalla.

Desarrollo

1. RAM vs. Disco

¿Por qué, si cerrás la ventana de consola y volvés a abrir el ejecutable, la Opción 2 sigue mostrando el historial de partidas anteriores?

Porque el historial **nunca estuvo en la RAM**. Al jugar una partida, el programa escribe los datos en un archivo físico del disco duro o unidad de estado sólido mediante `ofstream`. Un archivo en disco es **almacenamiento secundario**, es decir, almacenamiento **no volátil**: sus datos existen físicamente en el dispositivo y sobreviven a que el proceso termine.

La RAM, en cambio, es **volátil**: es un espacio de trabajo temporal. Cuando el programa finaliza, el sistema operativo libera ese bloque de memoria y su contenido se pierde definitivamente.

La demostración de persistencia de esta actividad sigue exactamente ese camino:

1. Se juega una partida (Opción 1) y se escribe en `src/historial_partidas.txt`.

2. Se cierra la ventana de consola por completo.

3. Se vuelve a abrir y se ejecuta el `.exe` de nuevo.

4. Se elige la Opción 2 y el historial sigue ahí.

Ese último paso es la prueba: el archivo sigue en disco y `ifstream` lo vuelve a leer. Si el historial hubiera estado solo en la RAM, la Opción 2 habría informado «Aun no existen partidas guardadas en disco».

2. Uso de `ios::app`

¿Qué sucedería con el archivo `historial_partidas.txt` si en la función `guardarEnDisco` se utilizara `ofstream archivo("historial_partidas.txt");` sin incluir la opción `ios::app`?

El archivo se **sobrescribiría por completo en cada partida nueva**, y el historial quedaría reducido a una única línea: la última partida jugada.

La razón es el **modo de apertura por defecto de `ofstream`**. Un flujo `ofstream` se abre en modo `ios::out` (salida), que es exactamente el modo que usa la función `fopen` con la bandera `w` (*write*): **crea el archivo si no existe y trunca todo su contenido si ya existe**. Es decir, el puntero de escritura arranca en la posición 0 del archivo, sobreescribiendo el primer byte, y cada escritura posterior va pisando lo que quedara.

El modo `ios::app` (*append*) cambia ese comportamiento: indica al sistema operativo que **agregue nuevos datos al final del archivo sin borrar el contenido existente**. El puntero de escritura arranca al final del archivo, de modo que cada partida se suma a las anteriores.

La diferencia observable es fácil de demostrar: jugando tres partidas con `ios::app` el archivo tiene tres registros; sin él, tiene uno.

Además, `ofstream` combinado con `ios::app` crea el archivo automáticamente si no existe. Por eso el archivo `historial_partidas.txt` **se crea la primera vez que se juega una partida** (Opción 1), y la Opción 2 avisa con «Aun no existen partidas guardadas en disco» si se consulta antes de eso.

3. Manejo del búfer (`cin.ignore()`)

¿Por qué es necesario invocar `cin.ignore()` antes de leer el nombre del jugador con `getline(cin, p.jugador)`?

Porque en el búfer de entrada hay un carácter de salto de línea (`\n`) pendiente de la lectura anterior.

La secuencia de la función es:

```
cout << "Ingrese su Nombre / Nickname: ";
cin.ignore();               // Descarta el Enter de "Ingrese una opcion (1-3): "
getline(cin, p.jugador);    // Ahora sí lee el nombre completo
```

El usuario elige la opción `1` en el menú principal y presiona Enter. El `cin >> opcion` consume el `1`, pero **el `\n` queda en el búfer de entrada**. Cuando se invoca `getline`, esa función lee caracteres **hasta encontrar el próximo `\n`**: encuentra inmediatamente el que quedó pendiente y devuelve una **cadena vacía**, sin darle al usuario ninguna oportunidad de escribir.

`cin.ignore()` descarta ese carácter pendiente y vacía el búfer, de modo que `getline` queda esperando una línea nueva. Sin esa instrucción, el síntoma es confuso: el programa **no falla**, simplemente muestra el nombre en blanco aunque el usuario lo haya escrito correctamente.

El mismo problema se repite si se usa `getline` después de cualquier `cin >>`, por lo que la instrucción es obligatoria y no opcional.

4. Agrupación con `struct`

¿Qué ventajas ofrece la estructura `Partida` para organizar las variables del juego frente a pasar 5 variables sueltas a cada función?

```
struct Partida
{
    int    idPartida;
```

```
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

La ventaja principal es la **cohesión de datos**: las cinco variables describen todos ellas **la misma cosa** —una partida de piedra, papel o tijera—. Al agruparlas en una sola entidad:

1. **Una función, un parámetro.** `guardarEnDisco(const Partida& p)` recibe un solo

argumento en lugar de cinco. Agregar un campo nuevo (por ejemplo, la fecha o la hora) requiere modificar una sola firma y un solo cuerpo, no cinco funciones.

2. **Integridad garantizada.** El compilador impide pasar el `idPartida` de una

partida junto con el `jugador` de otra. Con variables sueltas es perfectamente posible cruzarlas por error y no habría ninguna advertencia.

3. **Paso por referencia sin copias.** `const Partida& p` pasa la **dirección** de la

estructura. No se copia ninguna de las cinco variables en la RAM, que es especialmente relevante porque las dos variables `string` pueden apuntar a memoria dinámica cuya copia sería cara.

4. **Un bloque contiguo.** Las cinco variables quedan en casilleros consecutivos de

la RAM, lo que mejora el acceso a caché.

Además, la estructura es **serializable de forma directa**: como todos sus campos se escriben separados por comas en una misma línea, el archivo `.txt` generado es compatible con cualquier lector de **CSV**, lo que abre la puerta a una hoja de cálculo o a processing posterior en el Proyecto Integrador Anual.

5. Aplicación al P.I.A.

¿Qué datos o eventos del prototipo se guardarían en el archivo de texto?

Si el grupo tiene el P.I.A. avanzado, la estructura `Partida` puede adaptarse para registrar eventos reales del prototipo. La idea es exactamente la misma: cada evento es una entidad con un identificador, un actor, un valor de entrada, un valor de salida y un resultado.

Aplicación
idPartida
jugador
eleccionJugador
eleccionIA
resultado

Lecturas de sensores
Muestra n.º
Sensor (id)
Temperatura medida
Umbral configurado
Alarma / Normal

Aplicación idPartida jugador eleccionJugador eleccionIA resultado
Logs de usuarios Evento n.º Usuario Acción solicitada Acción ejecutada Permitida / Denegada
Estado de stock Movimiento n.º Producto Cantidad pedida Cantidad disponible Cubierto / Faltante
Fichaje de turnos Turno n.º Empleado Hora de entrada Hora registrada Puntual / Atrasado

En el caso del **Proyecto Integrador Anual**, la misma técnica permite guardar configuraciones o logs de hardware en un archivo `.txt` o `.csv`, de modo que el histórico del dispositivo sobreviva a un reinicio de la Raspberry Pi o del microcontrolador.

Tabla de control de revisiones (DocVer)

Versión Fecha Descripción Estado
`v1.0.0` 2026 Línea base del juego con persistencia en disco (`struct` + `fstream`). Menú de 3 opciones, registro en `historial_partidas.txt` y lectura del historial. **Baseline Aprobado**

Conclusión

La actividad resolvió de forma concreta la confusión entre RAM y disco, que es el error conceptual más común de los programadores que empiezan a trabajar con archivos.

Las conclusiones principales son cuatro. Primera: un programa que **no escribe en disco** simplemente no guarda nada, por muy bien programada que esté su lógica en memoria. Segunda: el **modo de apertura del archivo** (`ios::app` frente al modo `out` por defecto) decide entre acumular historial o destruirlo, y esa decisión cabe en un único parámetro. Tercera: el **búfer de entrada** es una fuente de errores silenciosos que solo se comprendiendo su comportamiento. Cuarta: el `struct` convierte cinco variables sueltas en una entidad coherente que se puede guardar, leer y transferir como una sola unidad.

La combinación `struct` + `fstream` es la base de cualquier sistema que necesite registrar eventos: desde un archivo de texto hasta una base de datos real. Dominar esta actividad deja preparado el terreno para el Proyecto Integrador Anual.

Referencias

• cppreference.com. (2024). `std::ofstream`. C++ reference. <https://en.cppreference.com/w/cpp/io/basic_ofstream>

• cppreference.com. (2024). `std::ifstream`. C++ reference. <https://en.cppreference.com/w/cpp/io/basic_ifstream>

• cppreference.com. (2024). `std::ios::app`. C++ reference. <https://en.cppreference.com/w/cpp/io/ios_base::open>

• Microsoft. (2024). *File input and output*. Microsoft Learn. <https://learn.microsoft.com/es-es/cpp/cpp/file-input-and-output>

• GeeksforGeeks. (2024). *C++ | std::getline (Function)*. <https://www.geeksforgeeks.org/getline-c-function/>

• Erichson, E. (2024). *Comma-separated values*. Wikipedia. <https://en.wikipedia.org/wiki/Comma-separated_values>